

Mémo projet — Mini SOC

Console de supervision et d'investigation léger inspirée des Security Operations Centers

Réalisateur : [À compléter — Nom, Prénom] · Formation : [À compléter] · Mai 2026

Synthèse

Mini SOC est une application web full stack qui simule les briques utiles dans un petit centre d'opérations sécurité (SOC) : collecte ou visualisation des **tentatives d'authentification**, création et suivi **d'alertes** (criticité, statut), **consultation des journaux**, **dashboard** agrégé, **export CSV**, **administration** des utilisateurs (rôles) et enrichissement IP optionnel (AbuseIPDB). L'accent est mis sur une **API REST sécurisée**, une **console React** ergonomique avec thème sombre type « poste analyste », et une **mise en conteneurs** permettant de rejouer l'environnement en groupe ou en soutenance.

Contexte et objectifs pédagogiques

Le projet mobilise plusieurs compétences typiques du développement d'applications d'entreprise :

- Concevoir une **architecture en couches** (API, persistance, présentation) avec séparation frontend / backend.
- Mettre en œuvre l'**authentification JWT** et des **droits différenciés** selon les rôles utilisateur (lecture auditor, analyste, actions admin).
- Orchestrer le **traitement événementiel** (succès ou échec de login → journalisation → règles de détection → alertes).
- Assurer la **qualité** (tests automatisés, analyse statique) et une **livraison reproductible** (Docker Compose, CI GitHub Actions).

Mini SOC reste une **sandbox pédagogique** : jeu de données de démonstration (fixtures), pas un SOC opérationnel connecté à un SI réel.

Choix techniques (stack)

Zone	Technologies
Backend	Symfony 7, API Platform (API REST / resources), Doctrine ORM, PostgreSQL

Zone	Technologies
Sécurité	Lexik JWT, voters Symfony pour les opérations sensibles
Frontend	React 19, Vite, Material UI, React Query (TanStack Query), Recharts & Data Grid
Données / cache	PostgreSQL ; Redis disponible dans la stack Compose (Messagerie / limiters selon configuration)
DevOps	Docker Compose (Nginx, PHP-FPM ou équivalent, frontend, PostgreSQL...), workflows GitHub Actions (PHPStan, PHPUnit, ESLint, Vitest, build)

Les échanges entre le frontend et le backend passent par l'API JSON ; la même origine peut être utilisée via Nginx sur le port 8080 (`/api`).

Fonctionnalités principales

Authentification et rôles

- Connexion par **email** / **mot de passe** ; jeton **JWT** pour les appels suivants.
- Contrôle d'accès selon les rôles (ex. niveaux **auditor** / **analyst** / **admin**) sur les lectures et mutations (alertes, exports, utilisateurs).

Logs et événements d'accès

- Enregistrement des tentatives (**succès**, **échec**, informations contextuelles) pour permettre audits et corrélations.
- **Rate limiting** côté point de login (`/api/login`) pour limiter le bruteforce sur l'entrée principale JSON.

Alertes et détection

- Génération et mise à jour d'**alertes** lorsque des règles de détection sont satisfaites (ex. comportements suspects sur la base des tentatives récentes).
- Attributs : type, gravité (**sévérité**), **statut** (parcours depuis la création jusqu'à clôture / assignation suivant le métier défini dans le projet).
- **Commentaires** sur alerte pour garder une trace d'investigation.

Dashboard et données agrégées

- Vue agrégée : volumes sur une fenêtre temps, alertes par sévérité, répartition de statuts, principales sources IP (graphiques avec Recharts).
- Endpoint **GET** `/api/dashboard/stats` (JWT, niveau auditor ou plus selon implémentation).

Exports

- Export **CSV** des journaux d'accès ou des alertes pour reporting ou analyse externe (rôles **analyst** et plus).

Administration

- Gestion minimale des utilisateurs / rôles pour la démo pédagogique (selon exposition API Platform et écrans prévus).

Enrichissement IP (optionnel)

- Intégration **AbuseIPDB** (clé configurable) avec cache / quota suivant implémentation actuelle dans la base PostgreSQL (**IpReputationCache**, quota journalier documenté dans le README du dépôt).
-

Interfaces utilisateur

- **Console web** centrée métier SOC : navigation claire entre Tableau de bord, Alertes, Détail alerte, Journaux, Profil et zone Admin selon les droits.
 - **Design** pensé comme poste analyste (**thème sombre**, typo lisible pour chiffres, cartes statistiques, graphiques lisibles dans un environnement peu lumineux).
-

Qualité logicielle et intégration continue

- **Backend** : PHPStan (`phpstan.neon`), tests PHPUnit fonctionnels/smoke où présents dans le projet.
 - **Frontend** : ESLint (style Airbnb projet), Vitest avec possibilité de couverture.
 - **CI** : fichier `.github/workflows/ci.yml` enchaîne ces contrôles sur les branches du dépôt pour limiter les régressions.
-

Déploiement et reproductibilité

- Déploiement local recommandé : **`docker compose up -d --build`** à la racine de `mini-soc/`, avec variables décrites dans les fichiers `.env.example` (PostgreSQL, secret applicatif, chemins JWT, etc.).
 - Compte démo après chargement des fixtures (voir README projet) permet une démonstration immédiate en soutenance.
-

Limites connues et prolongements possibles

- Données **synthétiques** et scénarios **limités** : il ne s'agit pas d'un SI production.
 - Pistes sérieuses pour la suite : intégration avec un **collecteur syslog** réel, règles de corrélation plus riches, playbook d'automations, notification **SIEM**/tickets externes, SOC multi-tenant, durcissement des politiques CSP et rotation des JWT.
-

Conclusion

Mini SOC synthétise un **parcours développement complet** : API moderne sous Symfony/API Platform, authentification stateless JWT, SPA React métier SOC, données relationnelles, conteneurs et pipeline CI **alignés avec les usages professionnels** tout en conservant une emprise raisonnable pour un rendu institutionnel ou une soutenance technique.